

REALTIME LOGS PROCESSING SYSTEM

Apache Airflow (MWAA) + Confluent Kafka + Elasticsearch

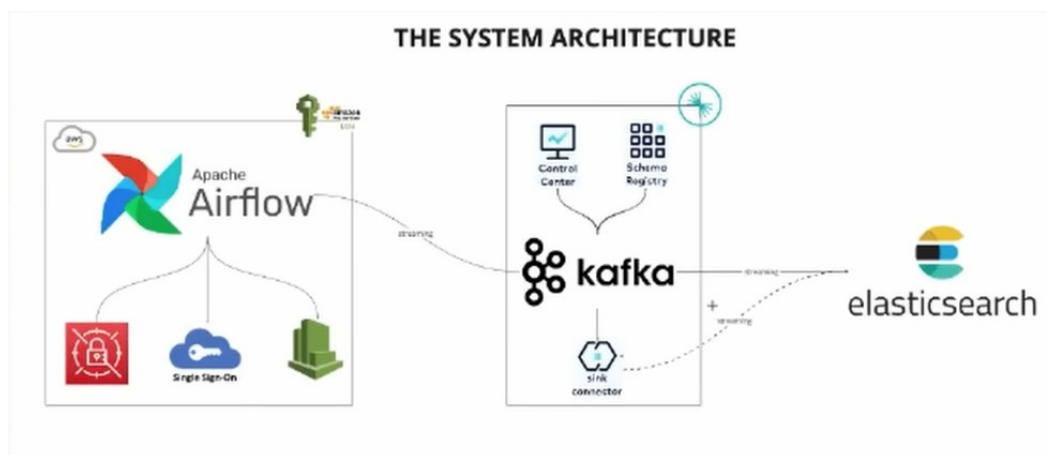
Complete End-to-End Implementation Notes with Production AWS Setup

1. PROJECT OVERVIEW

This project implements a **production-grade real-time logs processing pipeline** using modern cloud-native technologies. It demonstrates secure, scalable, and observable log ingestion, streaming, and analytics.

Goal: Generate/produce logs → Stream via Kafka → Index into Elasticsearch in near real-time, orchestrated by Amazon MWAA, with proper secret management and CI/CD.

2. ARCHITECTURE



Amazon MWAA Architecture

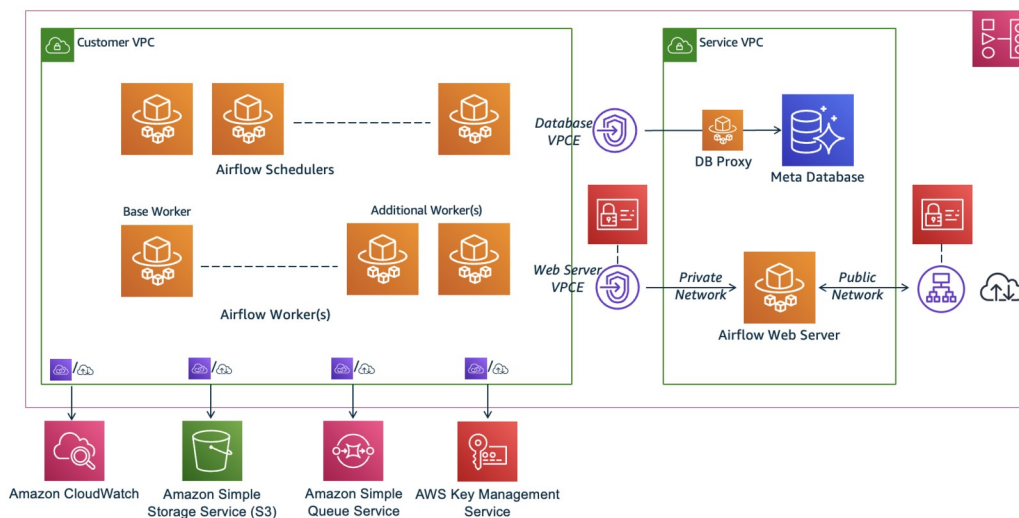


Figure 1: Complete System Architecture — Airflow (MWAA) orchestrates log generation & consumption pipelines with Confluent Kafka and Elasticsearch

3. TECHNOLOGY STACK (Production Implementation)

Layer	Technology	Details
Orchestration	Amazon MWAA	Managed Airflow 3.2.1 with S3 DAG sync via GitHub Actions
Streaming	Confluent Cloud Kafka	Fully managed Kafka with SASL/SSL authentication
Search & Analytics	Elasticsearch	Real-time indexing of logs (billion_website_logs index)
Secrets Management	AWS Secrets Manager	Secure storage of Kafka credentials (via MWAA Secrets)
Storage	Amazon S3	DAG code + versioned storage
CI/CD	GitHub Actions	Automatic sync of DAGs from GitHub → S3 → MWAA
Monitoring	Amazon CloudWatch + Airflow UI	Task logs, metrics, and pipeline observability

4. LIVE PRODUCTION IMPLEMENTATION (AWS Environment)

4.1 Amazon MWAA Environment

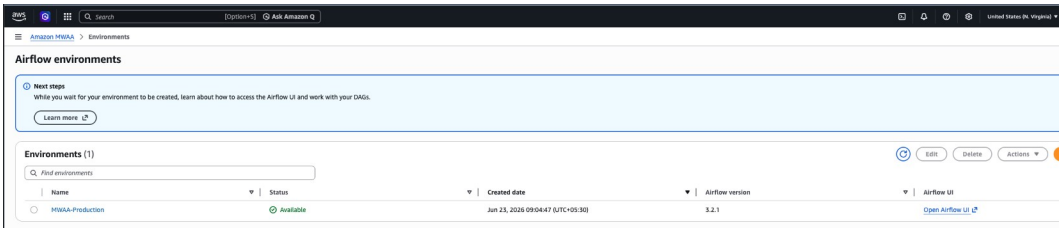


Figure 2: Your MWAA Production environment is healthy and running Airflow 3.2.1

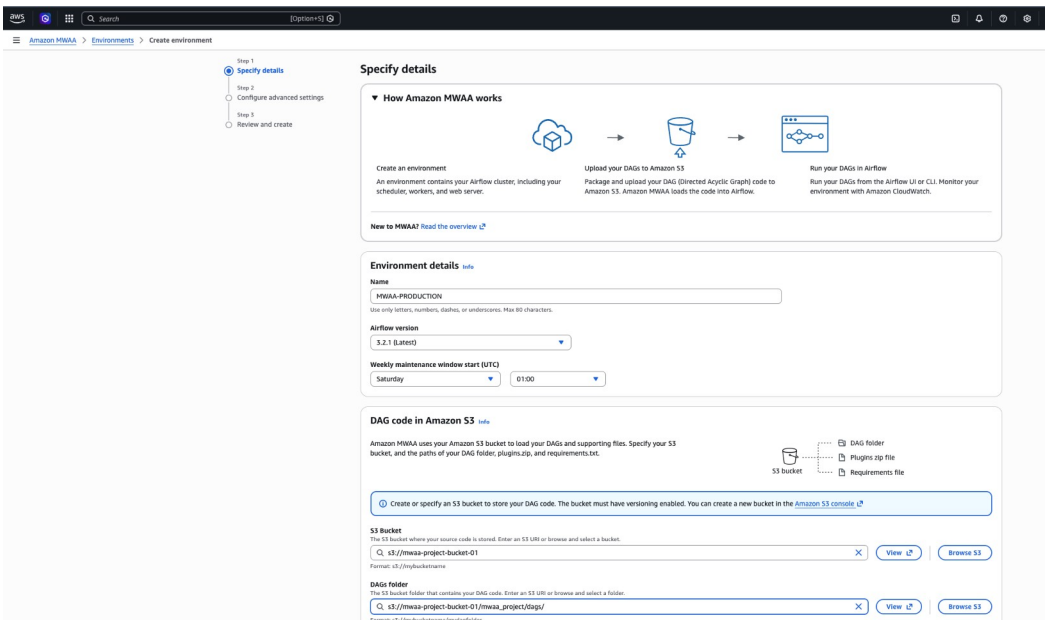


Figure 3: Amazon MWAA Architecture — DAGs are loaded from S3 and executed in the managed environment

4.2 Confluent Cloud Kafka

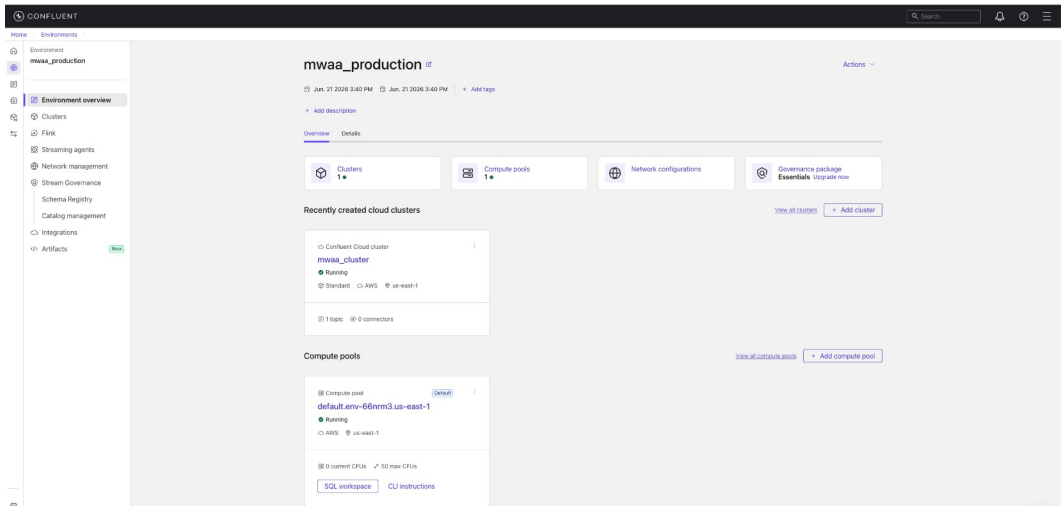


Figure 4: Confluent Cloud Kafka cluster (mwaa_production) running successfully

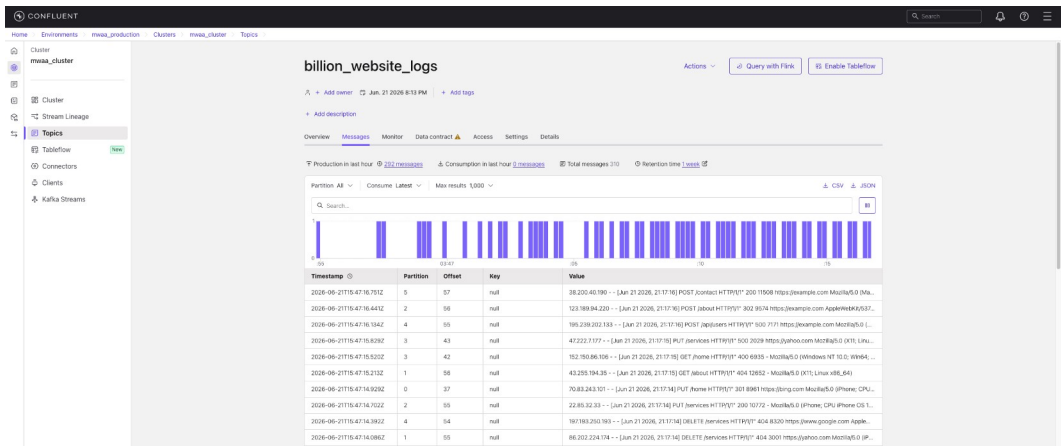


Figure 5: Kafka topic 'billion_website_logs' actively receiving messages in real-time

4.3 Airflow DAGs & Task Execution

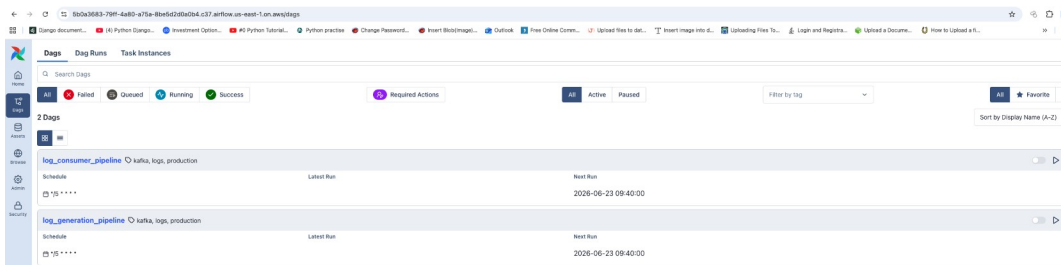
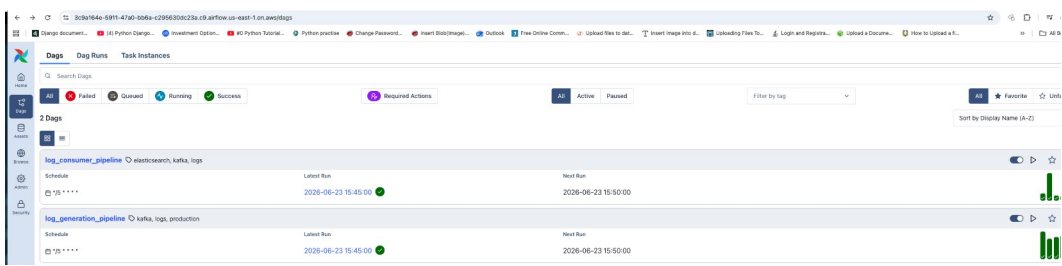


Figure 6: Two production DAGs — log_generation_pipeline and log_consumer_pipeline



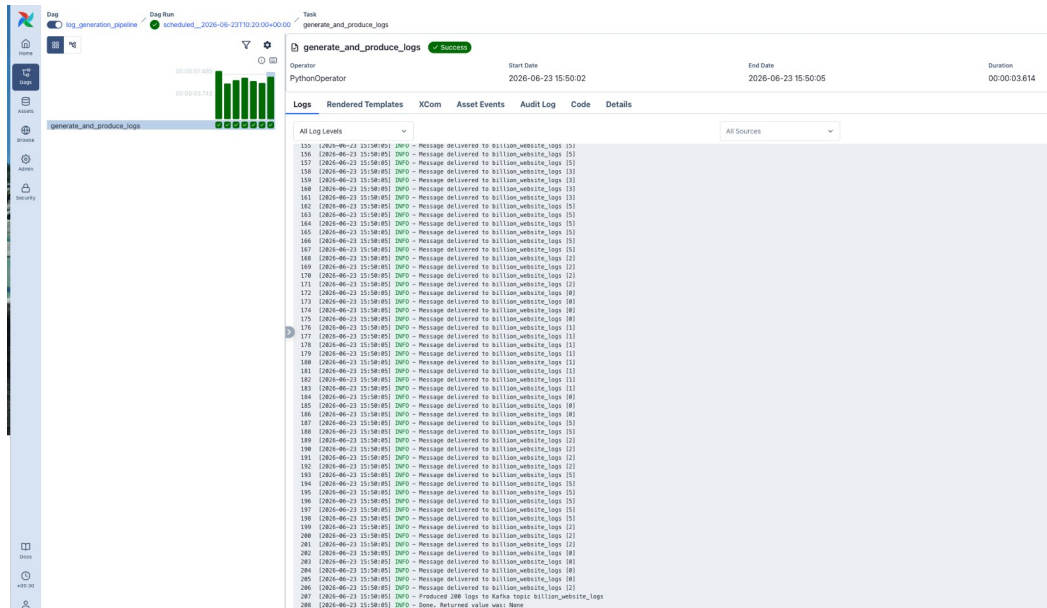


Figure 7: Successful execution of `generate_and_produce_logs` task with detailed Kafka delivery logs

4.4 Elasticsearch Integration

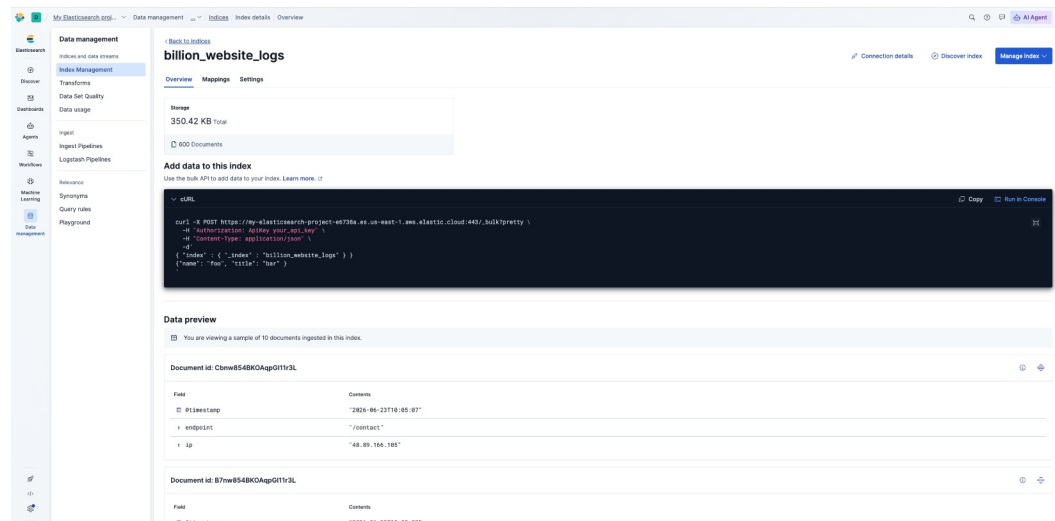
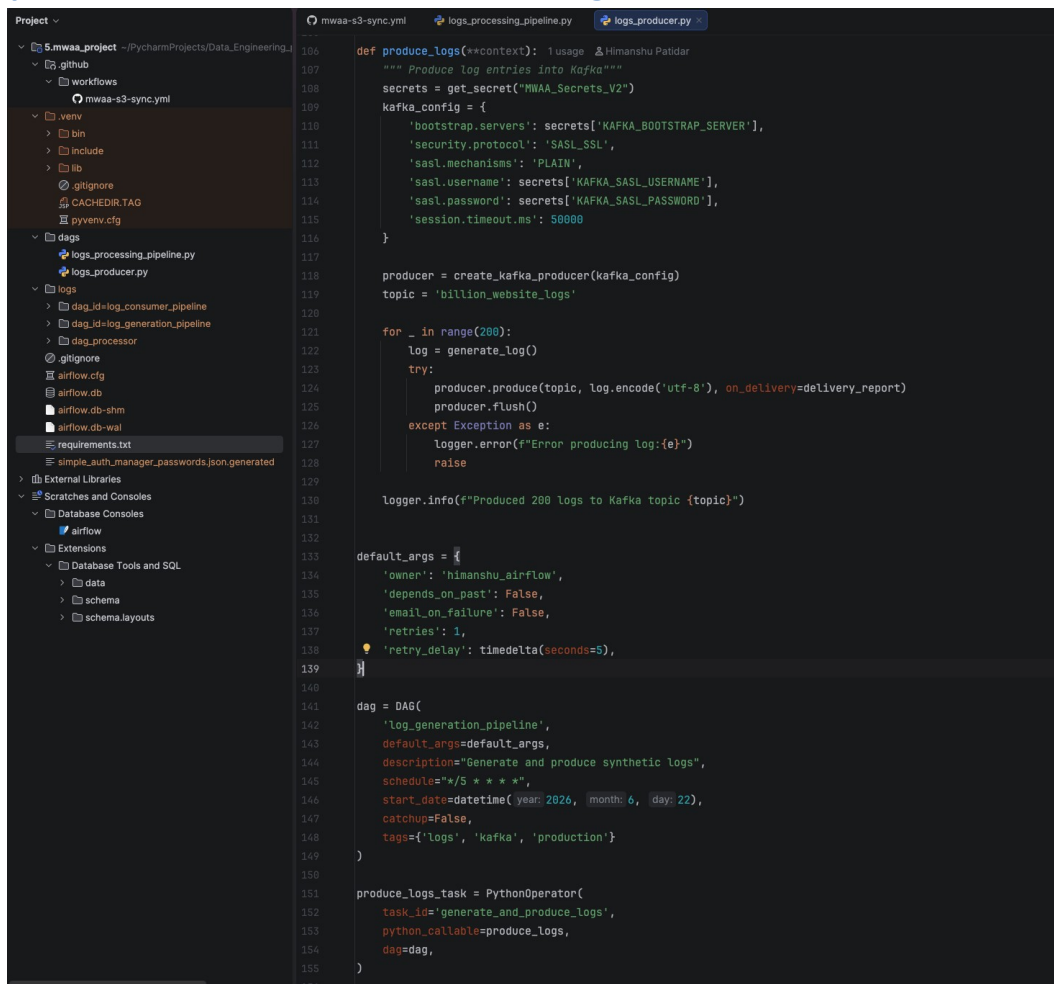


Figure 8: Elasticsearch index '`billion_website_logs`' receiving and storing logs in real-time

4.5 Code Implementation & Secure Credential Management



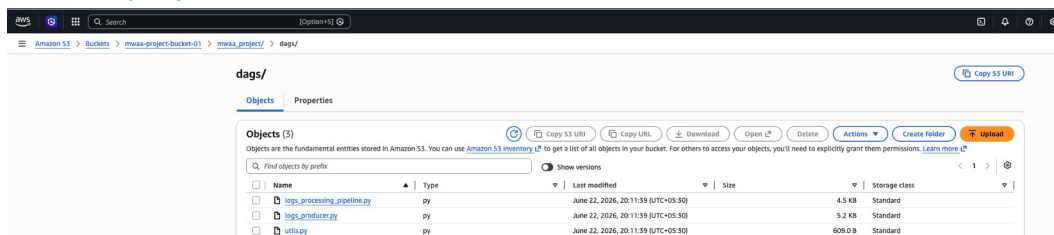
```
106 def produce_logs(**context): 1 usage 1 Himanshu Palidar
107 """ Produce log entries into Kafka"""
108 secrets = get_secret("MWAA_Secrets_V2")
109 kafka_config = {
110     'bootstrap.servers': secrets['KAFKA_BOOTSTRAP_SERVER'],
111     'security.protocol': 'SASL_SSL',
112     'sasl.mechanisms': 'PLAIN',
113     'sasl.username': secrets['KAFKA_SASL_USERNAME'],
114     'sasl.password': secrets['KAFKA_SASL_PASSWORD'],
115     'session.timeout.ms': 50000
116 }
117
118 producer = create_kafka_producer(kafka_config)
119 topic = 'billion_website_logs'
120
121 for _ in range(200):
122     log = generate_log()
123     try:
124         producer.produce(topic, log.encode('utf-8'), on_delivery=delivery_report)
125         producer.flush()
126     except Exception as e:
127         logger.error(f"Error producing log:{e}")
128         raise
129
130 logger.info(f"Produced 200 logs to Kafka topic {topic}")
131
132
133 default_args = {
134     'owner': 'himanshu_airflow',
135     'depends_on_past': False,
136     'email_on_failure': False,
137     'retries': 1,
138     'retry_delay': timedelta(seconds=5),
139 }
140
141 dag = DAG(
142     'log_generation_pipeline',
143     default_args=default_args,
144     description="Generate and produce synthetic logs",
145     schedule="*/5 * * * *",
146     start_date=datetime(year=2026, month=6, day=22),
147     catchup=False,
148     tags=['logs', 'kafka', 'production']
149 )
150
151 produce_logs_task = PythonOperator(
152     task_id='generate_and_produce_logs',
153     python_callable=produce_logs,
154     dag=dag,
155 )
```

Figure 9: Production-grade code — Using AWS Secrets Manager to securely fetch Kafka credentials (SASL/SSL) inside the producer task

This is a **critical production best practice** have been implemented correctly:

- Kafka credentials (bootstrap servers, SASL username/password) are **not hardcoded**.
- They are fetched securely at runtime using `get_secret("MWAA_Secrets_V2")` from **AWS Secrets Manager**.
- This follows AWS security best practices and makes the pipeline portable across environments.
- The producer uses `confluent_kafka` library with proper SASL_SSL configuration.

4.6 CI/CD & DAG Deployment



Name	Type	Last modified	Size	Storage class
logs_processing_pipeline.py	py	June 22, 2026, 20:11:39 (UTC+05:30)	4.5 KB	Standard
logs_producer.py	py	June 22, 2026, 20:11:39 (UTC+05:30)	5.2 KB	Standard
utils.py	py	June 22, 2026, 20:11:39 (UTC+05:30)	609.0 B	Standard

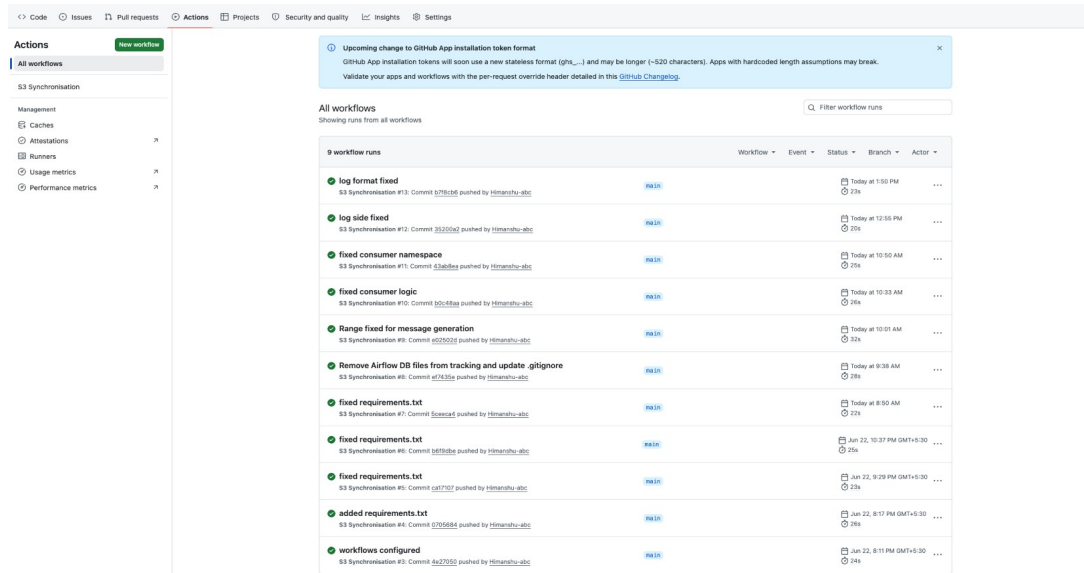


Figure 10: DAGs stored in Amazon S3 and automatically synced via GitHub Actions

5. HOW THE PIPELINE WORKS (End-to-End)

- 1. GitHub Actions pushes latest DAG code to Amazon S3
- 2. Amazon MWAA automatically picks up the latest code from S3
- 3. `log_generation_pipeline` runs on schedule → `'produce_logs'` task executes
- 4. Task securely fetches Kafka credentials from AWS Secrets Manager
- 5. 200 (dynamic) realistic logs are generated and published to Kafka topic `'billion_website_logs'`
- 6. `log_consumer_pipeline` consumes messages from Kafka and indexes them into Elasticsearch
- 7. Logs become immediately searchable in Elasticsearch
- 8. Full observability via Airflow UI + Amazon CloudWatch

6. KEY ACHIEVEMENTS

- Production-ready real-time logs pipeline on AWS (MWAA + Confluent + Elasticsearch)
- Secure credential management using AWS Secrets Manager (no hardcoded secrets)
- Proper CI/CD with GitHub Actions → S3 → MWAA
- Two clean, independent DAGs for generation and consumption
- Real-time message flow visible in both Kafka and Elasticsearch
- Excellent use of managed services for reduced operational overhead